FONAG — SEDC (Sistema de Estandarización de Datos Crudos)

Portal web de monitoreo hidrometeorológico, telemetría en tiempo real y anuarios estadísticos del **FONAG (Fondo para la Protección del Agua)**. Todos los datos provienen en vivo de la API Django del SEDC (`https://sedc.fonag.org.ec`) a través de funciones serverless propias; no hay datos locales de respaldo.

📋 Prerrequisitos de Entorno

- **Node.js**: versión 18 o superior (se recomienda v20 LTS). `package.json` no fija `engines` ; el mínimo lo impone Vite 6.
- **npm**: incluido con Node.js.
- **Git**: para el control de versiones.

🚀 Guía de Instalación y Levantamiento Local

1. Clonar el Repositorio

```
git clone git@github.com:webmasterft/fonag.git
cd fonag
```

2. Instalar Dependencias

Instala los paquetes (Vite 6, Tailwind CSS v4, Leaflet, Plotly, Chart.js, Embla Carousel, ESLint, Stylelint):

```
npm install
```

`npm install` ejecuta además el script `prepare` (`git config core.hooksPath .githooks`), que activa el hook **pre-commit** del repositorio (ver la sección 10 de la guía).

3. Configurar Variables de Entorno (`.env`)

Copia `.env.example` a `.env` en la raíz y completa los valores. El middleware de desarrollo de Vite y las funciones de Vercel inician sesión en SEDC con estas credenciales:

```
# URL base de la API oficial de SEDC (opcional; por defecto https://sedc.fonag.org.ec)
SEDC_API_BASE_URL=https://sedc.fonag.org.ec

# Credenciales SEDC (obligatorias)
SEDC_USERNAME=tu_usuario
SEDC_PASSWORD=tu_contrasena
```

> **Nota:** > Las credenciales son obligatorias y **solo** se leen de variables de entorno: no tienen valores por defecto en el código. Sin ellas, `npm run dev` muestra una advertencia al arrancar, las funciones responden `502` y cada página muestra el mensaje compartido "Error en el API, intenta más tarde.". No existen datasets locales de respaldo.
>
> Si la contraseña contiene `#` , escríbela entre comillas en `.env` (`SEDC_PASSWORD="..."`); de lo contrario dotenv la corta y el inicio de sesión falla.

En producción (Vercel) las mismas variables (`SEDC_USERNAME` , `SEDC_PASSWORD` y opcionalmente `SEDC_API_BASE_URL`) se configuran en **Project Settings** → **Environment Variables**; tras agregarlas hay que desplegar nuevamente.

4. Iniciar el Servidor de Desarrollo

```
npm run dev
```

La aplicación estará disponible en 👉 **`http://localhost:9000/`** (`server.port` en `vite.config.js`).

🛠️ Scripts Disponibles en `package.json`

Comando	Descripción
`npm run dev`	Servidor de desarrollo de Vite en el puerto `9000` con el middleware SEDC y HMR.
`npm run build`	Compila la aplicación Multi-Page (MPA) para producción en `dist/` .
`npm run preview`	Sirve localmente la compilación de `dist/` .
`npm run lint`	Ejecuta `lint:css` y luego `lint:js` .
`npm run lint:css`	Stylelint sobre `src/css/**/*.css` .
`npm run lint:js`	ESLint sobre `src/js` , `api/` , `scripts/` y `vite.config.js` .
`npm run check:secrets`	Revisa todos los archivos versionados en busca de credenciales escritas en el código.

El proyecto no tiene suite de pruebas automatizadas (no hay script `test`).

🗺️ Mapa de Rutas de la Aplicación (MPA)

Ruta	Descripción	Entrada HTML	Controller
`/`	Portada: mapa de ejes, "Cifras del Portal" y socios	`index.html`	`src/js/main.js`
`/estaciones/`	Catálogo de estaciones (tabla, filtros, columnas, CSV)	`estaciones/index.html`	`src/js/pages/estaciones.js`
`/consultas/periodo/`	Series de tiempo por periodo (figura y archivo oficial de SEDC)	`consultas/periodo/index.html`	`src/js/pages/periodo.js`
`/consultas/anuario/`	Anuario Hidroclimático (figuras, tablas y Excel oficial de SEDC)	`consultas/anuario/index.html`	`src/js/pages/anuario.js`
`/tiempo-real/`	Telemetría en tiempo real y estado de transmisión	`tiempo-real/index.html`	`src/js/pages/tiempo-real.js`
`/contacto/`	Formulario de contacto (envío a FormSubmit) y mapa embebido de la oficina	`contacto/index.html`	`src/js/pages/contacto.js`

📖 Guía Completa de Extensión e Implementación para Desarrolladores

1. Agregar una Nueva Página o Vista (Vite MPA)

1. **Crear el HTML**: crea una carpeta con su `index.html` (ej. `./reportes/index.html`) con `<header class="portal-header">` , `<main>` , `<footer>` y un único `<h1>` .
2. **Registrar la entrada en Vite**: añade el punto de entrada en `build.rollupOptions.input` de `./vite.config.js` :

   ```
   reportes: resolve(__dirname, 'reportes/index.html')
   ```

3. **Crear el controlador**: crea `./src/js/pages/reportes.js` e impórtalo en la vista:

   ```
   <script type="module" src="/src/js/pages/reportes.js"></script>
   ```

4. **Inicializar los componentes globales** en el controlador, como hacen las páginas existentes:

```
import { initPortalHeader } from '../organisms/portal-header.js';
import { initGlobalHttpLoader } from '../atoms/global-loader.js';
import { initThemeToggle } from '../organisms/theme-toggle.js';

document.addEventListener('DOMContentLoaded', () => {
  initPortalHeader();
  initGlobalHttpLoader();
  initThemeToggle();
  // ...lógica de la página
});
```

---

2. Convenciones de Estilos (CSS Tokens & Atomic Design)

Los estilos se organizan bajo **Atomic Design** en `./src/css/`:

- `tokens/` (`colors.css`, `typography.css`, `spacing.css`, agregados en `index.css`): color (`--palette-r{fila}-c{col}`, `--color-palette-*`, `--color-brand-*`), tipografía (`--font-*`), espaciado (`--space-*`), radios (`--radius-*`), sombras y transiciones.
- `atoms/`: botones (incluida `.btn-limpiar`), inputs, badges, sprites y `loader.css` (loader y estado de error de la API).
- `molecules/`: tarjetas de métricas, ítems de socios, datepicker, tablas, buscador.
- `organisms/`: header, hero, mapa de la portada, footer y layouts de cada vista.
- **Registro**: todo nuevo archivo `.css` debe importarse en `./src/css/main.css` respetando el orden de cascada.
- **Colores**: usa los tokens en código nuevo. El CSS existente aún contiene muchos hex escritos directamente; al tocar un archivo, preferir migrarlos a tokens.
- **Tema**: solo tema claro. `initThemeToggle()` elimina `data-theme` y la clave `fonag-theme` heredadas; no hay modo oscuro ni selector de tema.

---

3. Integración con la API SEDC

El cliente llama siempre a rutas propias `/api/sedc/*` (mismo origen). Esas rutas se resuelven así:

Entorno	Quién atiende `/api/sedc/*`
Desarrollo (`npm run dev`)	Middleware de `vite.config.js` (tabla `SEDC_ROUTES`), que ejecuta **las mismas funciones** de `api/*.js` que usa Vercel. Lee `SEDC_*` de `.env`.
Producción (Vercel)	`vercel.json` reescribe cada ruta a su Serverless Function en `api/`; todas comparten `api/_lib/sedc.js` (login Django + CSRF, caché de sesión de 30 min).

Catálogo de Endpoints

Servicio	Ruta del cliente	Función (`api/`)	Endpoint SEDC	Autenticación
Listado de estaciones	`GET /api/sedc/estacion/list/`	`estaciones.js`	`GET /estacion/list/`	Sesión (sin sesión SEDC omite `est_id` / `eje_trabajo`)
Catálogo de variables	`GET /api/sedc/variable/<seccion>/list`	`variables.js`	`GET /variable/<seccion>/list`	Sesión
Estaciones por variable	`GET /api/sedc/point_geojson`	— (rewrite directo a SEDC)	`GET /point_geojson`	Pública
Anuario hidroclimático	`POST /api/sedc/hydro_annual`	`hydro-annual.js`	`POST /hydro_annual`	Sesión + CSRF
Telemetría	`POST /api/sedc/telemetria/consulta`	`telemetria.js`	`POST /ajax/telemetria/consulta`	Sesión + CSRF
Estado de transmisión	`GET /api/sedc/telemetria/alarma_transmision`	`alarma-transmision.js`	`POST /ajax/telemetria/mapa_alarma_transmision`	Sesión + CSRF

Servicio	Ruta del cliente	Función (`api/`)	Endpoint SEDC	Autenticación
Consultas por periodo	`POST /api/sedc/reportes/consultas_periodo`	`periodo.js`	`POST /reportes/consultas_periodo`	Sesión + CSRF

Notas por endpoint:

1. **`/estacion/list/`**: parámetros permitidos `nombre`, `codigo`, `administrador`, `est_estado`, `order`, `limit`, `offset`, `year` (por defecto `administrador=1`, `limit=300`).
2. **`/variable/<seccion>/list`**: `seccion` solo admite letras y `_` (por defecto `hidro`); cualquier otro valor responde `400`. El cliente usa `hidro`.
3. **`/point_geojson`**: `section=hydroclimate&variable=<var_codigo>`; se usa para saber qué estaciones miden una variable.
4. **`/hydro_annual`**: body `{ stations: [filas completas de /estacion/list/], year, export? }`. Con `export: true` devuelve el Excel oficial.
5. **`/ajax/telemetria/consulta`**: body `{ estacion, inicio }`. SEDC **solo** acepta la fecha de inicio (no hay fecha de fin): Tiempo Real ofrece un único filtro "Fecha inicio" (por defecto, hoy).
6. **`/ajax/telemetria/mapa_alarma_transmision`**: sin parámetros; devuelve las estaciones con `estado` NORMAL / EXPECTANTE / FALLO y los `limites` en horas.
7. **`/reportes/consultas_periodo`**: body `{ estacion, variable, frecuencia, fecha_inicio, fecha_fin, export? }` con `frecuencia` `2`=Subhorario validado, `3`=Horario, `4`=Diario, `5`=Mensual, `6`=Anual. Con `export: true` devuelve el archivo oficial (CSV subhorario o Excel).

Las funciones reenvían a SEDC lo que envió el cliente (cadena vacía si falta), sin valores inventados. Para JSON envían `X-Requested-With: XMLHttpRequest`; para exportaciones lo omiten y pasan el archivo tal cual (`content-type` y `content-disposition` de Django). No envían cabeceras CORS. Sin credenciales o con login fallido responden `502`; con un método distinto, `405`.

Agregar un nuevo servicio SEDC

Ejemplo: exponer un endpoint autenticado `POST /reportes/nuevo_reporte` de SEDC.

1. **Función serverless** `api/nuevo-reporte.js`, usando los helpers de `api/_lib/sedc.js` (`sedcPost` para POST con CSRF, `sedcGet` para GET):

```
import { sedcPost, parseBody, sendJson, methodGuard } from './_lib/sedc.js';

/**
 * Vercel Serverless Function: nuevo reporte (POST /reportes/nuevo_reporte)
 * Route: /api/sedc/reportes/nuevo_reporte → /api/nuevo-reporte
 */
export default async function handler(req, res) {
  if (!methodGuard(req, res, 'POST')) return;
  try {
    const body = parseBody(req);
    // Forward only the known fields, without invented defaults
    const sedcResponse = await sedcPost('/reportes/nuevo_reporte', {
      estacion: String(body.estacion ?? ''),
    });
    sendJson(res, sedcResponse); // 502 when there is no SEDC session
  } catch (error) {
    console.error('[Vercel Nuevo Reporte Error]:', error);
    res.status(500).json({ error: error.message });
  }
}
```

2. **Rewrite de producción** en `vercel.json` (antes de cualquier regla más general):

```
{ "source": "/api/sedc/reportes/nuevo_reporte", "destination": "/api/nuevo-reporte" }
```

3. **Ruta de desarrollo** en `SEDC_ROUTES` de `vite.config.js` (importa el handler arriba del archivo):

```
import nuevoReporteHandler from './api/nuevo-reporte.js';
// ...
{ pattern: /^\/api\/sedc\/reportes\/nuevo_reporte\/?$/, handler: nuevoReporteHandler },
```

Si falta esta entrada, en desarrollo la ruta responde `404`.

4. **Servicio de cliente** en `src/js/services/` que llame a `/api/sedc/...` **sin respaldo**: si la respuesta no es `ok`, lanza un error para que la página muestre `showApiError()`:

```
export async function fetchNuevoReporte(estacion) {
  const res = await fetch('/api/sedc/reportes/nuevo_reporte', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ estacion: estacion.id }),
  });
  if (!res.ok) throw new Error(`Nuevo reporte no disponible (${res.status})`);
  return res.json();
}
```

5. Ejecuta `npm run lint` y prueba la ruta con `npm run dev` antes de desplegar.

Manejo de errores (sin respaldos)

Ante fallos de red o de autenticación no se usan datasets locales ni datos simulados: la página muestra el estado de error compartido de `src/js/atoms/api-error.js`:

```
import { showApiError } from '../atoms/api-error.js';
import { loadVariables, fetchSeriesDeTiempo, VARIABLES_CONFIG } from '../services/periodo-service.js';
import { renderPeriodoChart, buildStationChartTitle } from '../molecules/charts/periodo-charts.js';

try {
  await loadVariables(); // fills VARIABLES_CONFIG, required by fetchSeriesDeTiempo
  const series = await fetchSeriesDeTiempo(estacion, 'PRE', '2026-01-01', '2026-10-01', 'diario');
  renderPeriodoChart(canvas, series, VARIABLES_CONFIG.PRE, 'diario', { title: buildStationChartTitle(estacion) });
} catch (error) {
  showApiError(container); // "Error en el API, intenta más tarde."
}
```

- `fetchTelemetriaReal()` es la excepción: no lanza, devuelve `{ hasData: false, apiError: true }` cuando la API falla.
- Los valores individuales faltantes se muestran como `-` (`displayValue()`); nunca se reemplazan por valores por defecto.
- Si falla una descarga oficial (Periodo o Anuario) se muestra `alert(API_ERROR_MESSAGE)`; no se genera un archivo sustituto.
- Los contadores y cifras del HTML arrancan en `-` y se llenan con los datos de la API.

Datos geográficos locales (`ejes_2026.json`)

`./src/data/ejes_2026.json` es el único archivo de datos local: un GeoJSON con los polígonos de los **9 Ejes de Trabajo** (Antisana, Papallacta - Oyacachi, Pichincha Atacazo, Pita, San Pedro, Pisque, Norcentral, Noroccidente, Nororiente DMQ). Lo usan `leaflet-map.js` y `home-map.js` para dibujar los polígonos. No es un respaldo de la API: el eje de cada estación es `eje_trabajo` tal como lo devuelve SEDC y no se calcula en el cliente. Si se agrega o renombra un eje, actualiza a la vez el GeoJSON (`properties.eje_trab`), `EJES_CONFIG` y `formatEjeName()` en `leaflet-map.js`.

---

4. Seguridad en el DOM (`escape-html.js`)

Todo texto que viene de la API (nombres, códigos, opciones de selects, celdas, popups del mapa, títulos de gráficos) se inserta con `escapeHtml()` o `displayValue()` de `src/js/atoms/escape-html.js` antes de usar `innerHTML`. El markup estático propio no se escapa.

```
import { escapeHtml, displayValue } from '../atoms/escape-html.js';

card.innerHTML = `<span data-codigo="${escapeHtml(est.codigo)}">${displayValue(est.nombre)}</span>`;
```

---

5. Tablas y Exportación

No hay un motor genérico de tablas; cada página gestiona la suya.

- **Estaciones** (`src/js/pages/estaciones.js`): filtrado, orden y paginación en el controlador; columnas visibles con `initColumnFilter()` (`src/js/molecules/data-table/column-filter.js`); CSV generado en el cliente con las estaciones recibidas de la API.
- **Periodo**: archivo oficial de SEDC con `downloadPeriodoExport()` (CSV subhorario validado o Excel).
- **Anuario** (`src/js/molecules/charts/anuario-sedc.js`): tablas mensuales con la respuesta de `/hydro_annual` ; Excel oficial de SEDC con `downloadAnuarioExcel()` , por estación o compilado de las estaciones filtradas.
- **Tiempo Real**: SEDC no exporta telemetría, así que "Descargar datos" genera en el cliente un CSV con las lecturas reales recibidas (`exportarSerieCsv()` de `periodo-service.js`).
- Los botones de descarga de los modales quedan ocultos (`hidden`) cuando no hay datos.

---

6. Gráficos (Plotly + Chart.js como render alternativo)

1. **Plotly (principal)**: Periodo dibuja la figura `grafico` que devuelve SEDC; Anuario, las figuras `graph` de `/hydro_annual` ; Tiempo Real construye la figura con `buildTelemetriaFigure()` como en Django. Plotly se carga de forma diferida con `loadPlotly()` de `src/js/atoms/plotly-loader.js` .
2. **Chart.js**: `renderPeriodoChart()` solo lo usa cuando la serie no trae `series.figure` .
3. **Título estándar**: usa `buildStationChartTitle(estacion)` (`periodo-charts.js`), que produce "Estación {código} {nombre}" escapado.
4. Destruye siempre la instancia previa (`destroyPeriodoChart()` / `destroyAnuarioSedc()`) antes de dibujar otra.

---

7. Mapas (Leaflet)

- `src/js/molecules/map/leaflet-map.js` (`initLeafletMap`) es el mapa de Anuario, Periodo y Tiempo Real; `src/js/organisms/home-map.js` es el de la portada.
- Ambos usan la configuración exportada por `leaflet-map.js` (`EJES_CONFIG` , `TYPE_COLORS`) y los polígonos de `ejes_2026.json` .
- **Select "Eje de trabajo" sincronizado**: en Anuario y Tiempo Real, hacer clic en un polígono actualiza el select (`onEjeSelect`) y cambiar el select enfoca el polígono (`mapController.setEje()`). En Periodo el select de eje es un filtro adicional e independiente del eje del mapa.
- Las estaciones vienen de `fetchEstaciones()` ; los popups escapan todos los datos de la API. Las estaciones sin coordenadas se listan pero no se dibujan.

---

8. Componentes Globales

- **Loaders (`src/js/atoms/global-loader.js`)**: `initGlobalHttpLoader()` envuelve `window.fetch` y muestra la barra de progreso y la tarjeta "Consultando servidor..." en peticiones a `/api/` , URLs absolutas o `.json` ; `showInlineLoader(container)` muestra el mismo loader dentro de un contenedor (modales, mapa, gráfico) y devuelve la función para quitarlo. Estilos en `src/css/atoms/loader.css` .
- **Error de la API (`src/js/atoms/api-error.js`)**: `API_ERROR_MESSAGE` , `apiErrorHtml()` , `showApiError()` .
- **Escape de HTML (`src/js/atoms/escape-html.js`)**: `escapeHtml()` , `displayValue()` .
- **Botón "Limpiar"**: clase `.btn-limpiar` de `src/css/atoms/buttons.css` en todas las vistas con filtros.
- **Cifras del Portal (`src/js/organisms/portal-stats.js`)**: estaciones, estaciones con telemetría, variables y año inicial, calculados con la API (reutiliza la petición de estaciones del mapa de la portada).
- **Estado de transmisión (`fetchEstadoTransmision()`)**: colorea "Último Dato" en Tiempo Real; si falla, queda neutro.
- **Tema (`src/js/organisms/theme-toggle.js`)**: fuerza el tema claro.

---

9. Accesibilidad

- Modales de Periodo, Anuario y Tiempo Real: `role="dialog"` , `aria-modal="true"` , `aria-labelledby` , botón de cierre con `aria-label` y cierre con `Escape` . No hay gestión ni trampa de foco.
- Filtros con `<label for>` en las vistas de consulta y Estaciones. El formulario de Contacto usa solo `placeholder` .
- Portada y Contacto no tienen `<h1>` ; las demás vistas tienen uno. Las vistas nuevas deben tener un único `<h1>` .

10. 🔒 Control de credenciales (pre-commit)

- `.githooks/pre-commit` ejecuta `node scripts/check-secrets.js` sobre las líneas añadidas en stage y bloquea el commit si detecta credenciales escritas en el código: variables `SEDC_*` con valor por defecto (`|| '...'` / `?? '...'`), contraseñas, tokens o API keys literales y URLs con usuario y clave. Se permiten placeholders como `tu_usuario` .
- El hook se activa con `npm install` (script `prepare`). Para revisar todo el repositorio: `npm run check:secrets` .
- `.env` está ignorado por git; `.env.example` solo debe contener placeholders.

---

11. Metodología de Trabajo y Commits (SDD & Git)

1. **Spec-Driven Development (SDD)**: antes de cambios complejos, documenta la especificación (`openspec/changes/` contiene artefactos de cambios anteriores).
2. **Conventional Commits**: `feat:` , `fix:` , `docs:` , `style:` , `refactor:` , `perf:` , `chore:` (con scope opcional, p. ej. `fix(tiempo-real): ...`).
3. **Sin firmas de IA**: no añadas pies de página tipo `Co-Authored-By` ni notas automáticas de herramientas de IA.
4. **Rama principal**: `main` (remoto `origin` : `git@github.com:webmasterft/fonag.git`). Antes de integrar: `npm run lint` y `npm run build` .

🏗️ Arquitectura Técnica Resumida

- **Arquitectura**: Multi-Page Application (MPA) con **Vite 6** y **Vanilla JS** (ES modules).
- **Backend propio mínimo**: Serverless Functions de Vercel en `api/` (+ `api/_lib/sedc.js`), reutilizadas por el middleware de desarrollo de `vite.config.js` .
- **Gráficos**: Plotly (figuras de SEDC) con Chart.js solo como render alternativo.
- **Diseño**: CSS Tokens + Atomic Design (`tokens/` , `atoms/` , `molecules/` , `organisms/` , `templates/`).
- **Datos 100% SEDC**: sin datasets de respaldo; ante fallos se muestra el error compartido.

Para la documentación técnica detallada, lee ARCHITECTURE.md.